

GitHub Repository Governance

Repository Structure Improvements

Current Structure

```
greenfield_mp/
├── .devcontainer/
├── .git/
├── app/
│   ├── __init__.py
│   ├── streamlit_app.py
│   └── views/
│       ├── __init__.py
│       ├── dashboard.py
│       └── data_entry.py
├── docs/
│   ├── ARCHITECTURE.md
│   ├── GIT_WORKFLOW.md
│   └── diagrams/
│       ├── oltp_er_diagram.mmd
│       └── olap_star_schema.mmd
├── scripts/
│   ├── _common.py
│   ├── load_staging.py
│   ├── run_etl.py
│   ├── setup_database.py
│   ├── smoke_test.py
│   └── synthesize_data.py
├── sql/
│   ├── 01_schemas_and_staging.sql
│   ├── 02_oltp_ddl.sql
│   ├── 03_olap_ddl.sql
│   ├── 04_staging_to_oltp.sql
│   ├── 05_dw_etl_procedures.sql
│   └── 06_analytics_queries.sql
├── src/
│   ├── __init__.py
│   ├── db_manager.py
│   ├── entities.py
│   └── managers.py
├── README.md
└── requirements.txt
```

Recommended Improvements

1. **Standardize directory naming:** Use consistent naming conventions (all lowercase, hyphens for separation)
2. **Separate concerns more clearly:** - `data/` for raw and processed data files - `infrastructure/` for deployment scripts and configurations - `tests/` for all test files - `notebooks/` for exploratory analysis
3. **Add missing directories:** - `tests/unit/` for unit tests - `tests/integration/` for integration tests - `tests/performance/` for performance tests - `data/raw/` for source data -

`data/processed/` for cleaned data - `data/warehouse/` for warehouse-specific data -
`infrastructure/docker/` for containerization - `infrastructure/ci/` for CI/CD configurations

4. **Standardize file extensions:** - Use `.sql` for all SQL files - Use `.py` for all Python files -
Use `.md` for all documentation - Use `.yaml` or `.yml` for configuration files - Use `.json` for
JSON configuration

Branching Strategy

Main Branches

- `main` : Production-ready code, always deployable
- `develop` : Integration branch for features, used for testing before release

Feature Branches

- `feature/*` : New features or enhancements
- Examples: `feature/user-authentication` , `feature/dashboard-charts`
- Branch off from: `develop`
- Merge back to: `develop` via Pull Request

Bug Fix Branches

- `bugfix/*` : Fixes for bugs in current release
- Examples: `bugfix/login-issue` , `bugfix/data-validation`
- Branch off from: `main` or `develop` (depending on where bug exists)
- Merge back to: `main` (if critical) and `develop`

Hotfix Branches

- `hotfix/*` : Critical fixes for production issues
- Examples: `hotfix/security-patch` , `hotfix/database-connection`
- Branch off from: `main`
- Merge back to: `main` and `develop`

Release Branches

- `release/*` : Preparation for production release
- Examples: `release/v1.0` , `release/v2.1`
- Branch off from: `develop`
- Merge back to: `main` and `develop` after release

Branch Protection Rules

Main Branch Protection

1. **Require pull request reviews before merging** - Minimum 1 approving review - Dismiss stale approvals when new commits are pushed
2. **Require status checks to pass before merging** - All required checks must pass - Include CI/CD pipeline, tests, and security scans
3. **Include administrators** - Rules apply to repository administrators as well
4. **Restrict who can push to matching branches** - Only allow specific teams or individuals to push directly - Prefer pull request workflow for all changes
5. **Require linear history** - Prevent merge commits, require rebasing or squashing
6. **Allow force pushes** - Disable force pushes to prevent history rewriting
7. **Require conversation resolution before merging** - All comments must be resolved before merge

Develop Branch Protection

Similar to main but with slightly relaxed rules: 1. Require pull request reviews (minimum 1) 2. Require status checks to pass 3. Include administrators 4. Allow force pushes (for feature branch updates) 5. Do not require linear history (allows merge commits)

Feature/Bugfix/Hotfix Branches

No branch protection rules required, but recommended: 1. Require status checks for CI 2. Allow force pushes (for rebasing during development)

Pull Request Workflow

PR Creation

1. Create branch from appropriate base (`develop` for features, `main` for hotfixes)
2. Make changes and commit frequently with descriptive messages
3. Push branch to remote repository
4. Open Pull Request targeting appropriate base branch

PR Description Template

```
## Summary
Brief description of changes made.

## Related Issue
Closes #issue_number

## Type of Change
- [ ] Bug fix
- [ ] New feature
```

```
- [ ] Breaking change
- [ ] Documentation update
- [ ] Refactor
- [ ] Test update

## Checklist
- [ ] Code follows project style guidelines
- [ ] Self-review completed
- [ ] Tests added or updated
- [ ] Documentation updated
- [ ] No breaking changes (or breaking changes documented)

## Screenshots (if applicable)
![Before](link)
![After](link)

## Testing Performed
List of tests performed and results.

## Deployment Notes
Any special deployment instructions or considerations.
```

PR Review Process

1. **Automated Checks** - CI/CD pipeline runs on PR creation and updates - Includes linting, unit tests, integration tests, and security scans - All checks must pass before manual review
2. **Manual Review** - At least one approving review required - Reviewers should check:
 - Code quality and adherence to standards
 - Test coverage and quality
 - Documentation completeness
 - Security implications
 - Performance impact
3. **Review Comments** - Use GitHub's review system for line-specific comments - General feedback in PR comments - Request changes if necessary - Approve when all concerns addressed
4. **Merging** - After approval and all checks passing - Use squash and merge for feature/branch commits - Use merge commit for release branches - Delete branch after merging

Code Review Requirements

What to Review

1. **Functionality** - Does the code solve the intended problem? - Are edge cases handled? - Is the logic correct and efficient?

2. **Code Quality** - Follows project coding standards - Proper naming conventions - Appropriate comments and documentation - No duplicated code - Proper error handling
3. **Testing** - Adequate test coverage - Tests are meaningful and well-written - Both positive and negative test cases - Mocks used appropriately
4. **Security** - No hardcoded secrets - Proper input validation - Protection against common vulnerabilities (SQL injection, XSS, etc.) - Secure use of dependencies
5. **Performance** - Efficient algorithms and data structures - Proper database query optimization - Caching where appropriate - No N+1 query problems
6. **Documentation** - Clear README and inline documentation - Updated API documentation if applicable - Architecture diagrams updated if needed

Review Checklist

- ☐ Code compiles/runs without errors
- ☐ Follows PEP 8 (Python) or equivalent standards
- ☐ Meaningful variable and function names
- ☐ Appropriate use of data structures
- ☐ Proper error handling and logging
- ☐ No commented-out code or debug statements
- ☐ Tests added/modified as needed
- ☐ Documentation updated
- ☐ Security considerations addressed
- ☐ Performance implications considered
- ☐ Backward compatibility maintained (if applicable)

Merge Strategy

Preferred Methods

1. **Squash and Merge** - Default for feature branches - Creates clean, linear history - Combines all commits into single meaningful commit - Commit message should summarize the entire feature
2. **Merge Commit** - Used for release branches - Preserves branch history - Clearly shows when features were integrated - Use `--no-ff` to ensure merge commit is created
3. **Rebase and Merge** - Alternative for feature branches when linear history is critical - Rebases feature branch onto target before merging - Requires force push if branch was already pushed

When to Use Each

- **Squash and Merge**: Most feature development, small changes, documentation updates

- **Merge Commit:** Release branches, major features, when preserving branch history is important
- **Rebase and Merge:** When you want linear history but don't want to lose individual commit details (less common)

Commit Message Guidelines

- Use imperative mood ("Add feature" not "Added feature")
- Keep subject line under 50 characters
- Body should explain why, not what (what is visible in diff)
- Reference related issues: "Closes #123" or "Related to #456"
- Separate subject from body with blank line
- Use bullet points for multiple changes in body

Issue Templates

Bug Report Template

```

---
name: Bug report
about: Create a report to help us improve
title: '[BUG] '
labels: bug
assignees: ''

---

**Describe the bug**
A clear and concise description of what the bug is.

**To Reproduce**
Steps to reproduce the behavior:
1. Go to '...'
2. Click on '....'
3. Scroll down to '....'
4. See error

**Expected behavior**
A clear and concise description of what you expected to happen.

**Screenshots**
If applicable, add screenshots to help explain your problem.

**Environment (please complete the following information):**
- OS: [e.g. iOS]
- Browser [e.g. chrome, safari]
- Version [e.g. 22]

**Additional context**
Add any other context about the problem here.

```

Feature Request Template

```
---
name: Feature request
about: Suggest an idea for this project
title: '[FEATURE] '
labels: enhancement
assignees: ''

---

**Is your feature request related to a problem? Please describe.**
A clear and concise description of what the problem is. Ex. I'm always frustrated when
[...]

**Describe the solution you'd like**
A clear and concise description of what you want to happen.

**Describe alternatives you've considered**
A clear and concise description of any alternative solutions you've considered or tried.

**Additional context**
Add any other context or screenshots about the feature request here.
```

Documentation Request Template

```
---
name: Documentation request
about: Suggest improvements to documentation
title: '[DOC] '
labels: documentation
assignees: ''

---

**What documentation needs improvement?**
Please specify which documentation files or sections need updates.

**Why is this improvement needed?**
Explain the reason for the requested documentation changes.

**Suggested content**
Provide the suggested improvements or additions to the documentation.

**Additional context**
Add any other context or screenshots about the documentation request here.
```

PR Templates

Default Pull Request Template

```
## Summary
[Provide a brief summary of the changes]

## Related Issue
[Closes #issue_number] or [Related to #issue_number]
```

```
## Type of Change
- [ ] Bug fix
- [ ] New feature
- [ ] Breaking change
- [ ] Documentation update
- [ ] Refactor
- [ ] Test update

## Checklist
- [ ] Code follows project style guidelines
- [ ] Self-review completed
- [ ] Tests added or updated
- [ ] Documentation updated
- [ ] No breaking changes (or breaking changes documented)

## Screenshots (if applicable)
[Add screenshots or GIFs to demonstrate the changes]

## Testing Performed
[Describe the testing performed and results]

## Deployment Notes
[Any special deployment instructions or considerations]
```

Hotfix PR Template

```
## Summary
[Brief description of the critical fix]

## Related Issue
[Closes #issue_number] - This is a production hotfix

## Type of Change
- [x] Bug fix
- [ ] New feature
- [ ] Breaking change
- [ ] Documentation update
- [ ] Refactor
- [ ] Test update

## Checklist
- [ ] Code follows project style guidelines
- [ ] Self-review completed
- [ ] Tests added or updated
- [ ] Documentation updated
- [ ] No breaking changes (or breaking changes documented)
- [ ] This fix addresses a critical production issue
- [ ] Smoke tested in staging environment

## Screenshots (if applicable)
[Add screenshots if relevant]

## Testing Performed
[Describe testing performed - should include smoke test, regression test, etc.]

## Deployment Notes
[Include specific deployment instructions for hotfix - may require immediate deployment]
```


Release PR Template

```
## Summary
[Summary of release contents]

## Related Issue
[Related to epic or milestone if applicable]

## Type of Change
- [ ] Bug fix
- [ ] New feature
- [ ] Breaking change
- [ ] Documentation update
- [ ] Refactor
- [ ] Test update
- [x] Release preparation

## Checklist
- [ ] Code follows project style guidelines
- [ ] Self-review completed
- [ ] Tests added or updated
- [ ] Documentation updated
- [ ] No breaking changes (or breaking changes documented)
- [ ] All features for this release are complete
- [ ] Version number updated in appropriate files
- [ ] Changelog updated

## Screenshots (if applicable)
[Add screenshots if relevant]

## Testing Performed
[Describe comprehensive testing performed for release]

## Deployment Notes
[Include deployment instructions and rollback plan]
```